Software Engineering Research Group
MSc Thesis Style

Version of April 13, 2026

Leon Moonen

Software Engineering Research Group MSc Thesis Style

THESIS

submitted in partial fulfillment of the
requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Leon Moonen
born in Weert, the Netherlands

Software Engineering Research Group
Department of Software Technology
Faculty EEMCS, Delft University of Technology
Delft, the Netherlands
www.ewi.tudelft.nl

Some Company
With it's address
ThePlace, the Netherlands
www.url.nl

© 2026 Leon Moonen. *Note that this notice is for demonstration purposes and that the LaTeX style and document source are free to use as basis for your MSc thesis.*

Cover picture: A “random” maze generated in postscript.

Software Engineering Research Group MSc Thesis Style

Author: Leon Moonen
Student id: 123456789
Email: Leon.Moonen@computer.org

Abstract

This document describes the standard thesis style for the Software Engineering department at Delft University of Technology. The document and it's source are an example of the use of the standard LaTeX style file. In addition the final appendix to this document contains a number of requirements and guidelines for writing a Software Engineering MSc thesis.

Your thesis should either employ this style or follow it closely.

Thesis Committee:

Chair:	Prof. Dr. A. van Deursen, Faculty EEMCS, TU Delft
University supervisor:	Dr. E. Visser, Faculty EEMCS, TU Delft
Company supervisor:	Drs. E.X. Ternal, Some Company
Committee Member:	Dr. S.T.A.F.F. Member, Faculty EEMCS, TU Delft

Preface

This is where you thank people for helping you etc.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Phasellus massa pede, feugiat sit amet, mollis in, sodales at, augue. Ut sit amet nisi egestas risus consequat adipiscing. Nulla non diam. Proin volutpat, lacus quis volutpat scelerisque, leo urna rhoncus arcu, vel ultrices dui lacus id lorem. Nam pulvinar adipiscing odio. Etiam tellus lorem, malesuada in, scelerisque sit amet, consequat a, tellus. Curabitur non urna. Mauris facilisis tempor nulla. Nam euismod semper massa. Nullam id nulla. Duis mattis nunc ut ipsum. Proin libero purus, posuere ut, tincidunt sit amet, accumsan sit amet, nisl. Integer commodo. Pellentesque suscipit, diam vel bibendum interdum, magna mauris venenatis lorem, vitae tristique nibh lacus convallis velit. Sed tellus. Mauris placerat lectus ut tellus rutrum blandit. Aliquam erat volutpat.

Suspendisse potenti. Proin sodales eros non lacus. Nam magna sapien, tristique ut, hendrerit ultricies, pretium ut, ante. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Nulla facilisi. In libero risus, pellentesque vitae, interdum id, tincidunt ut, sapien. Mauris nec massa sit amet leo dictum pretium. Curabitur iaculis euismod mauris. Donec diam sem, pulvinar at, luctus id, blandit nec, pede. Nam scelerisque sollicitudin nunc. Nam malesuada mauris id ligula. Donec suscipit posuere justo. Mauris sed libero in mi nonummy tincidunt.

Leon Moonen
Delft, the Netherlands
April 13, 2026

Contents

Preface **iii**

Contents **v**

List of Figures **vii**

1 Introduction **1**
1.1 Research Question(s) 1

2 Background & Theory **3**
2.1 Operator Learning 3
2.2 Fourier Neural Operator 3
2.3 Convolutional Neural Operator 4
2.4 Mesh-free Operator Learning and geo-FNO 4
2.5 Discretization invariance on mesh-free grids 5

3 Experimental Setup **7**
3.1 Datasets 7
3.2 Training setup 11
3.3 Uniform-grid resolution transfer experiments 12
3.4 Non-uniform-grid experiments 12

4 Results **13**
4.1 Part I: Discretization Invariance Study 13
4.2 Part II: Non-uniform Meshes and geo-FNO 14
4.3 Writing note 15

5 Conclusions and Future Work **17**
5.1 Contributions and Conclusions 17
5.2 Future work 17

Bibliography **19**

A Disclosure on AI use **21**

B Navier-Stokes data generation details **23**

C HTGR input engineering details **25**

D Hyperparameters **27**

E Requirements and Guidelines **29**
E.1 Requirements 29
E.2 Guidelines 30

List of Figures

3.1 Initial and final vorticity fields for a sample from the Navier-Stokes dataset. . . 8

3.2 Example of an input diffusion coefficient field $a(x,y)$ and the resulting pressure field $u(x,y)$ with Dirichlet boundary conditions. 8

3.3 Example initial and final states for the incompressible flow past a cylinder dataset, shown on the irregular geometry together with the underlying point cloud. The color scale represents the speed $\|\mathbf{v}\|$. 10

3.4 Example temperature field from the HTGR column dataset for a fixed boundary temperature and selected axial layer, shown together with the irregular spatial mesh used for sampling. 11

B.1 Fourier spectra of an example initial and final vorticity field from the generated Navier-Stokes dataset. The initial spectrum reflects the truncated low-frequency initialization with Gaussian weighting, while the final spectrum shows the redistribution of spectral energy over time. 23

Chapter 1

Introduction

PDEs are very important in many scientific and engineering domains etc etc etc classical solvers expensive learning-based surrogates can perform similarly but produce answers in a fraction of the time standard machine learning approaches are vector to vector, we want function to function define discretization invariance and how it relates to neural operators/function to function architectures include research questions and motivation for the work, and a brief summary of the contributions of the thesis. The introduction should provide a clear overview of the problem being addressed, the significance of the research, and how the thesis is structured to address the research questions.

1.1 Research Question(s)

The research question investigated in this thesis is ...

Chapter 2

Background & Theory

This chapter introduces the theoretical background needed for the remainder of the thesis. It first places operator learning in context with more classical learning setups and then discusses the neural operator architectures that are central to this work.

2.1 Operator Learning

This section introduces operator learning as a mapping between functions, rather than the more familiar vector-to-vector setting. It should explain why this perspective is useful for PDE-governed physical systems and introduce the CDE concept that motivates resolution-aware learning.

2.1.1 Vector-to-vector versus function-to-function mapping

Contrast conventional supervised learning with operator learning and explain why PDE solution operators are naturally formulated in the function-to-function setting.

2.1.2 Continuous-domain perspective

Introduce the continuous-domain view and connect it to the notion of discretization invariance used throughout the thesis.

2.2 Fourier Neural Operator

This section should explain the FNO architecture and the spectral convolution mechanism on which it is based.

2.2.1 Architecture

Describe the lifting layer, Fourier layers, spectral convolution, non-linear transformations, and projection back to the target field.

2.2.2 Strengths

Discuss the global receptive field, computational efficiency through FFT-based convolutions, and the suitability of FNO for smooth solution operators.

2.2.3 Weaknesses

Discuss spectral truncation, sensitivity to resolution changes, and the reliance on regular grids.

2.3 Convolutional Neural Operator

This section should explain the CNO architecture and the inductive biases that differentiate it from FNO.

2.3.1 Architecture

Describe the encoder-decoder style structure, convolutional blocks, and the way multi-scale processing is used.

2.3.2 Bandlimited assumption

Explain the bandlimited assumption and why it can stabilize learning and support more consistent behavior across discretizations. Discuss how this means explicitly discarding high-frequency information, where the FNO keeps it (but does not know what to do with it)

2.3.3 Local versus global inductive bias

Compare the locality of convolutional operators with the global spectral bias of FNO and connect this to the later experiments.

2.4 Mesh-free Operator Learning and geo-FNO

This section motivates operator learning on irregular domains and introduces geo-FNO as the architecture used for this purpose.

2.4.1 Motivation for irregular domains

Explain why many scientific and engineering problems do not naturally live on uniform Cartesian grids and why this matters for neural operators.

2.4.2 geo-FNO

Describe the geo-FNO approach and how it extends Fourier-based operator learning to non-uniform meshes.

2.5 Discretization invariance on mesh-free grids

More philosophical discussion of what discretization invariance means when we no longer have a regular grid and how this relates to the notion of a continuous-domain operator.

Chapter 3

Experimental Setup

This chapter describes the datasets and training procedures used throughout the thesis. It also clarifies how models with different output-resolution behavior are compared in the discretization transfer experiments.

3.1 Datasets

The experiments in this thesis use both uniform-grid and non-uniform datasets. For each dataset, the goal of this section is to describe the underlying physical problem, the mapping learned by the model, the data source, and the aspects of the dataset that are relevant for interpreting the results. Dataset-specific implementation details are included only when they affect the conclusions drawn later in the thesis. In particular, the full generation procedure for the self-generated Navier-Stokes dataset is deferred to Appendix B. For all datasets, 80% of the samples were used for training and 20% for testing, unless otherwise noted.

3.1.1 Uniform-grid datasets

The discretization invariance experiments on regular grids use two datasets: a self-generated Navier-Stokes dataset and a Darcy flow dataset obtained from prior work.

Navier-Stokes

For the regular-grid experiments, we use a self-generated two-dimensional Navier-Stokes dataset on the unit square. The Navier-Stokes equations are among the fundamental equations of fluid dynamics and arise in many areas of physics and engineering, including atmospheric and oceanic flows, aerodynamics, and industrial transport processes. The dataset is formulated and solved in vorticity form, with governing equation:

$$\partial_t \omega + u \cdot \nabla \omega = \nu \Delta \omega + f, \tag{3.1}$$

where ω denotes the vorticity field, u the incompressible velocity field, ν the viscosity, and f an external forcing term. The learning task considered in this thesis is not a pressure or velocity prediction problem, but an operator mapping from an initial vorticity field to

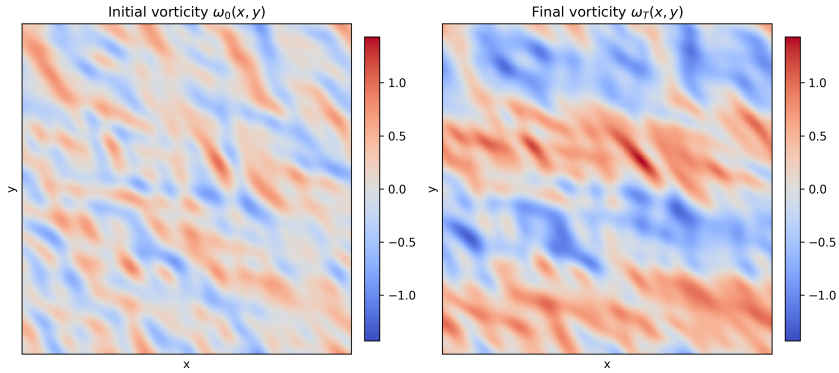

Figure 3.1: Initial and final vorticity fields for a sample from the Navier-Stokes dataset.

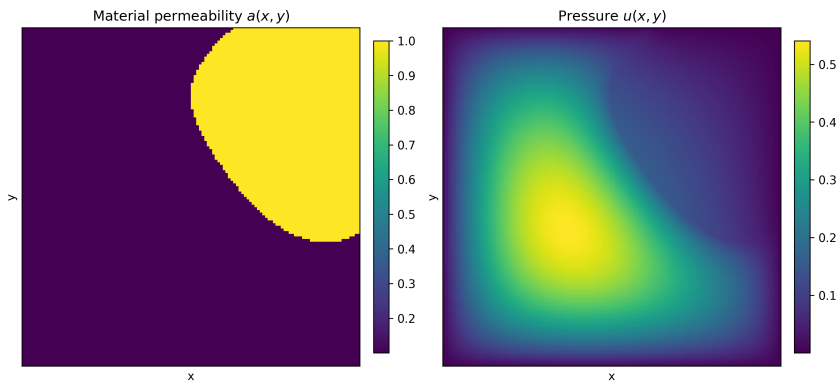

Figure 3.2: Example of an input diffusion coefficient field $a(x, y)$ and the resulting pressure field $u(x, y)$ with Dirichlet boundary conditions.

a later vorticity field. The input to the model is the initial vorticity field $\omega(\cdot, 0)$, and the output is the vorticity field at a later time $\omega(\cdot, T)$. An example input-output pair is shown in Figure 3.1.

The dataset consists of 1000 samples generated at a resolution of 256×256. Between samples, the viscosity and forcing remain fixed, while the initial vorticity field is varied.

Further details on the data-generation procedure are provided in Appendix B.

Darcy flow

Also used for the uniform grid experiments is the Darcy flow dataset introduced by Li et al.[7]. In this benchmark, the task is to learn the solution operator of a steady-state two-dimensional Darcy flow problem on the unit square, governed by the following PDE:

$$-\nabla \cdot (a(x)\nabla u(x)) = f(x), \qquad x \in (0,1)^2, \tag{3.2}$$

$$u(x) = 0, \qquad x \in \partial(0,1)^2, \tag{3.3}$$

The input is a spatially varying diffusion coefficient field $a(x)$, and the output is the corresponding solution field $u(x)$ under a prescribed forcing term and Dirichlet boundary conditions. An example input-output mapping can be seen in figure 3.2. Between samples, $f(x)$ is kept constant while the $a(x)$ is varied. This PDE has many applications within physics and engineering, including drainage and irrigation in agricultural soils and fluid migration around nuclear waste repositories [8]. For this thesis, the Darcy flow dataset serves as a regular-grid benchmark for evaluating the performance of FNO and CNO across discretizations. This dataset was selected because it provides a comparatively smooth benchmark, complementing the more irregular and turbulent behavior of the Navier-Stokes dataset discussed in Section 3.1.1. Using both datasets makes it possible to evaluate model performance under qualitatively different solution regimes. The dataset contains 10000 samples at a resolution of 128×128.

3.1.2 Non-uniform datasets

The experiments on irregular domains primarily use two datasets: an incompressible Euler dataset modeling flow past a cylinder and a nuclear engineering dataset consisting of temperature profiles for a single HTGR column.

Elasticity

The elasticity dataset is included because it is one of the canonical benchmarks used to evaluate geo-FNO on non-uniform meshes. It provides a direct connection to the original geo-FNO setting and therefore helps assess whether the implementation used in this thesis reproduces the expected behavior on an established test case.

In the dataset description, it is enough to explain the physical problem at a high level, describe the input and output quantities, and note that the data is defined on an irregular geometry rather than a uniform Cartesian grid. The main text should also cite the original source and report the experimental resolution or mesh representation at the level needed to interpret the results.

Should I inlclu

Incompressible Navier-Stokes flow past a cylinder

A second non-uniform benchmark is an incompressible Navier-Stokes dataset describing fluid flow past a cylinder. This dataset was used in prior work on operator learning for PDEs and is included here because it provides a fluid-dynamics test case on a geometry that is not naturally aligned with a regular grid [6, 9].

The dataset describes two-dimensional incompressible flow with velocity $\mathbf{v}$ and pressure p, governed by:

$$\nabla \cdot \mathbf{v} = 0, \tag{3.4}$$

$$\rho_0 \left(\partial_t \mathbf{v} + \mathbf{v} \cdot \nabla \mathbf{v}\right) + \nabla p = \mu \nabla^2 \mathbf{v}, \tag{3.5}$$

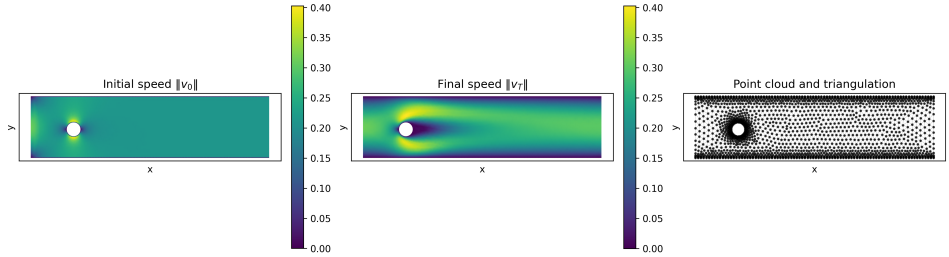

Figure 3.3: Example initial and final states for the incompressible flow past a cylinder dataset, shown on the irregular geometry together with the underlying point cloud. The color scale represents the speed $\|\mathbf{v}\|$.

where ρ_0 is the constant density and μ the viscosity. The domain consists of a channel containing a cylindrical obstacle, with the cylinder position and diameter varying between samples.

The prediction task is to learn a mapping from the velocity field at the initial time to the velocity field at the final time, while accounting for the fact that changes in cylinder size and location lead to different corresponding flow patterns. In the experiments of this thesis, the model is trained to predict only the horizontal and vertical velocity components; pressure is not included in the target.

This dataset is particularly relevant for geo-FNO because it is defined on a non-uniform spatial discretization rather than a regular Cartesian grid. An example initial-final pair is shown in Figure 3.1.2. The dataset consists of 1000 samples each containing between 1700 and 2100 points, depending on the geometry of the sample. 200 of these samples were designated for testing, while the remaining 800 were used for training.

HTGR column temperature dataset

The final non-uniform dataset consists of steady-state temperature profiles for a single column of a high temperature gas-cooled reactor (HTGR). This dataset is included because it provides a first application-oriented test case for geo-FNO in a nuclear engineering setting. Unlike the public benchmark datasets considered earlier, the data was produced internally as part of ongoing work by a colleague.

Physically, the dataset describes the temperature distribution within a three-dimensional reactor column. The spatial sampling locations lie on an irregular mesh that is shared across all samples. Along the axial direction, the column is quantized into 17 distinct z-levels. The boundary temperature ranges from 300 °C to 1000 °C in steps of 5 °C, yielding 141 distinct thermal conditions. Since the prediction task is defined on individual horizontal slices rather than on the full three-dimensional column, this results in a total of 2397 two-dimensional samples.

In the prediction task considered in this thesis, the model receives the boundary temperature together with a specified z-level and predicts the temperature at all sampled (x, y) locations in the corresponding horizontal slice. The task can therefore be interpreted as

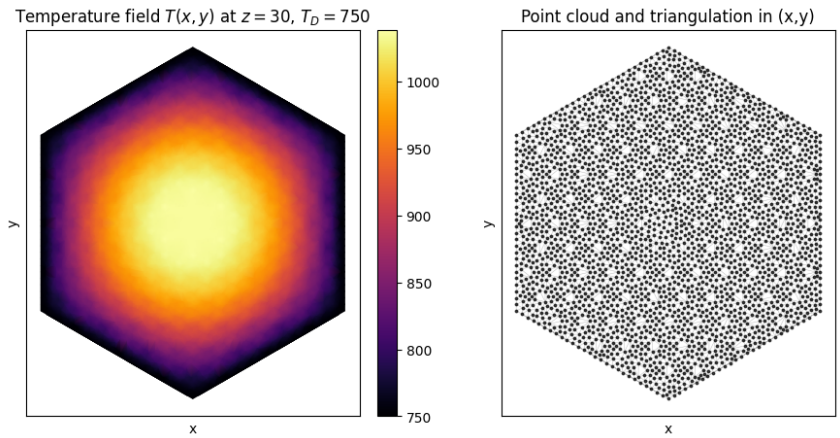

Figure 3.4: Example temperature field from the HTGR column dataset for a fixed boundary temperature and selected axial layer, shown together with the irregular spatial mesh used for sampling.

learning a mapping from the boundary condition and axial position to the two-dimensional temperature profile within that layer. An example of such a slice, together with the underlying irregular mesh, is shown in Figure 3.4.

The train-validation-test split is constructed by excluding specific boundary temperatures and specific z-levels from training, rather than by applying a conventional random split. As a result, the held-out data contains samples with unseen temperatures, unseen axial levels, or both, making the evaluation more demanding by explicitly testing the model's ability to generalize across both thermal conditions and spatial positions.

Since the model input is limited to the axial level z and the boundary temperature T, the raw representation is relatively simple and may not be sufficiently expressive for learning the underlying mapping. For this reason, feature engineering was used to construct an expanded representation of the input variables before training. Feature engineering refers to the transformation or augmentation of raw input variables into a representation that is more informative for the learning algorithm, and prior work has shown that such feature engineering can improve predictive performance [5]. The exact form of this representation, as well as a comparison of model performance across different input encodings, is presented in Appendix C.

3.2 Training setup

All models are trained in a supervised setting on paired input-output samples from the datasets described above. For most datasets, the data is split into a training set and a held-out evaluation set. The training set is used to optimize the model parameters, while the held-out set is used to monitor performance during training and to report final model performance. For the HTGR column temperature dataset, a separate train-validation-test split is used,

constructed by holding out specific boundary temperatures and axial levels as described above.

The exact hyperparameters differ between the uniform-grid and non-uniform-grid experiments and between datasets. To keep the main text focused on the experimental design rather than implementation details, full hyperparameter settings, including learning rates, batch sizes, training epochs, and architecture parameters, are given in Appendix D.

3.3 Uniform-grid resolution transfer experiments

The uniform-grid experiments are designed to evaluate discretization invariance by testing how well a model trained at one spatial resolution generalizes to other resolutions. For each dataset, the model is trained at a chosen training resolution and then evaluated both at the training resolution and at lower and higher resolutions. This makes it possible to compare performance at the training resolution with performance after transferring the same trained model to different spatial resolutions.

FNO and CNO handle changes in resolution differently. FNO produces predictions of the same resolution as the input it is given, so the model output can be compared directly to the target field at that resolution. CNO, however, produces predictions on its internal resolution. Therefore, when CNO is evaluated on an input resolution different from its internal resolution, the CNO prediction is resampled onto the input resolution before the error is computed. This resampling step is included so that FNO and CNO are evaluated against target fields on the same spatial grid.

3.4 Non-uniform-grid experiments

The non-uniform-grid experiments evaluate geo-FNO on datasets whose spatial sampling locations do not lie on a regular Cartesian grid. In these experiments, the model is trained and tested on the same underlying point cloud or mesh associated with each dataset, so no grid-based resampling step is required when computing the prediction error.

Because the point cloud is fixed within each non-uniform dataset, these experiments do not test discretization invariance in the same resolution-transfer sense as the uniform-grid experiments. REFER TO SECTION DISCUSSING DISCRETIZATION INVARIANCE AND ITS EXTENSION/CONNECTION TO NON-UNIFORM GRIDS. Instead, they test whether geo-FNO can learn mappings on irregular spatial data and generalize to held-out samples, boundary conditions, time steps, axial levels, or material configurations, depending on the dataset.

Chapter 4

Results

This chapter presents the empirical findings of the thesis. It is split into two parts: a study of discretization invariance for FNO and CNO, and an investigation of geo-FNO on non-uniform meshes, including its application to a nuclear dataset.

4.1 Part I: Discretization Invariance Study

4.1.1 FNO versus CNO

Experimental design

Describe the setup in which each model is trained on one grid resolution and evaluated on unseen resolutions.

Results

Present the quantitative comparison between FNO and CNO and discuss how performance changes under resolution transfer.

4.1.2 Interpretation of discretization effects

This section should connect the observations back to the theoretical differences between the models.

FNO

Discuss spectral truncation and the fact that Fourier mode index does not preserve the same physical meaning when the discretization changes.

CNO

Discuss the role of locality, the bandlimited assumption, and the implicit multiscale structure in producing steadier behavior across resolutions.

Connection to theory

Tie the empirical behavior back to the contrast between bandlimiting and Fourier truncation.

4.2 Part II: Non-uniform Meshes and geo-FNO

4.2.1 Results

Present the geo-FNO results on the irregular-domain datasets and discuss the overall performance.

4.2.2 Analysis of seam artifacts

This section can document the seam artifact, explain how it manifests in different datasets, and analyze why some datasets are more robust to it than others.

Observed artifact

Describe the seam, the associated corruption of low-k_x modes, and the evidence that it appears across datasets.

Tracing the source

Explain how the issue was traced to the `conv0` layer and what was ruled out during debugging, including the approximate `fft2d`/`ifft2d` functions and the handling of complex weights.

Patch and effect

Explain the bug in the original implementation, how it was fixed, and show the before-and-after effect of the patch.

4.2.3 Nuclear application

Describe the nuclear dataset, explain the prediction task, and discuss how geo-FNO performs on unseen z levels and temperatures.

Dataset characteristics

Explain why the dataset is relatively simple and consider supporting that claim with a PCA visualization.

Generalization results

Discuss the extent to which the experiment demonstrates that geo-FNO extends to nuclear applications.

4.3 Writing note

An open presentation choice is whether to show the original seam-affected plots in the main text before introducing the patch, or to present only the corrected results in the main text and move most debugging evidence to an appendix.

Chapter 5

Conclusions and Future Work

This chapter gives an overview of the project's contributions. After this overview, we will reflect on the results and draw some conclusions. Finally, some ideas for future work will be discussed.

5.1 Contributions and Conclusions

showed that CNO is more robust to resolution changes than FNO, and connected this to the bandlimited assumption and the way that Fourier truncation does not preserve the same physical meaning across resolutions. Showed that geo-FNO can be applied to non-uniform meshes and performs well on the irregular-domain datasets, but also identified a seam artifact that appears across datasets and can corrupt low-k_x modes. Showed that geo-FNO can be applied to nuclear datasets.

split into two sections I think; decide structure once rest of thesis is written.

5.2 Future work

talk about possible improvements, shortcomings, what I would have done differently knowing what i know now etc.

Bibliography

[1] G. K. Batchelor. *An Introduction to Fluid Dynamics*. Cambridge University Press, Cambridge, 1967.

[2] Gary J. Chandler and Rich R. Kerswell. Invariant recurrent solutions embedded in a turbulent two-dimensional kolmogorov flow. *Journal of Fluid Mechanics*, 722:554–595, 2013. doi: 10.1017/jfm.2013.122.

[3] Elliot J. Chikofsky and James H. Cross II. Reverse engineering and design recovery: A taxonomy. *IEEE Software*, 7(1):13–17, 1990. doi: 10.1109/52.43044.

[4] Alexandre J. Chorin and Jerrold E. Marsden. *A Mathematical Introduction to Fluid Mechanics*. Springer, New York, 3 edition, 1993.

[5] Aditi Gulati, Armin Felahatpisheh, and Camilo E. Valderrama. Feature engineering through two-level genetic algorithm. *Machine Learning with Applications*, 21:100696, 2025. ISSN 2666-8270. doi: https://doi.org/10.1016/j.mlwa.2025.100696. URL https://www.sciencedirect.com/science/article/pii/S2666827025000799.

[6] Jan Hagnberger, Daniel Musekamp, and Mathias Niepert. Calm-pde: Continuous and adaptive convolutions for latent space modeling of time-dependent pdes, 2025. URL https://arxiv.org/abs/2505.12944.

[7] Zongyi Li, Nikola B. Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *CoRR*, abs/2010.08895, 2020. URL https://arxiv.org/abs/2010.08895.

[8] Abdou Njifenjou. Some models of darcy flows in porous media, 07 2023.

[9] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. Learning to simulate complex physics with graph networks, 2020. URL https://arxiv.org/abs/2002.09405.

[10] Alberto Vela-Martín and Marc Avila. Large-scale patterns set the predictability limit of extreme events in kolmogorov flow. *Journal of Fluid Mechanics*, 986:A2, 2024. doi: 10.1017/jfm.2024.263.

[11] Colin Wheildon. *Type and Layout: How Typography and Design Can Get Your Message Across, or Get in the Way*. Strathearn Communication Consultants, Sydney, 1995.

Appendix A

Disclosure on AI use

AI-based tools were used during this thesis project as support for coding, debugging, troubleshooting, source finding, and comprehension of technical material. During the writing process, they were also used to help structure sections and improve phrasing.

All AI-generated suggestions were reviewed and edited before being included. The research decisions, experiments, interpretation of results, and final thesis text remain my own responsibility.

Appendix B

Navier-Stokes data generation details

This appendix provides additional details on the generation of the self-produced Navier-Stokes dataset used in the uniform grid experiments. The dataset is formulated in vorticity form on the unit square and is generated numerically from the incompressible Navier-Stokes equation [1, 4]:

$$\partial_t \omega + u \cdot \nabla \omega = \nu \Delta \omega + f, \tag{B.1}$$

where ω denotes the vorticity field, u the incompressible velocity field, ν the viscosity, and f an external forcing term. The velocity field is not part of the stored dataset or the prediction target. Instead, it is reconstructed from the vorticity during data generation in order to evaluate the nonlinear advection term, using the streamfunction relation [1, 4]:

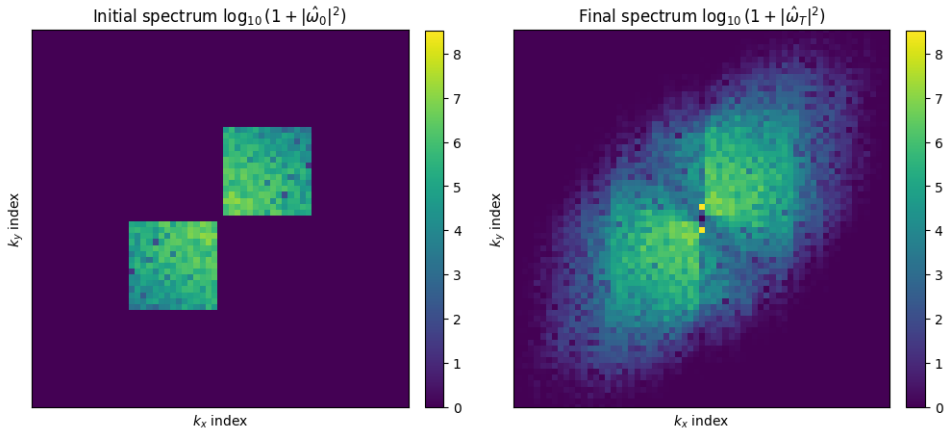

Figure B.1: Fourier spectra of an example initial and final vorticity field from the generated Navier-Stokes dataset. The initial spectrum reflects the truncated low-frequency initialization with Gaussian weighting, while the final spectrum shows the redistribution of spectral energy over time.

$$-\Delta\psi = \omega, \qquad u = \nabla^{\perp}\psi. \tag{B.2}$$

In all generated samples, the viscosity is kept fixed at $\nu = 10^{-5}$. The forcing term is also shared across samples and is given by:

$$f(y,t) = 0.5\,(1 + 0.5\sin(0.5t))\sin(4\pi y), \tag{B.3}$$

which is consistent with the Kolmogorov-flow literature, where the forcing is commonly chosen as a single sinusoidal mode that varies only in one spatial direction [2, 10]. In the present dataset, the spatial factor $\sin(4\pi y)$ serves as the corresponding single-mode forcing on the unit square. The prefactor was chosen so that the forcing remains smooth, bounded, and of moderate magnitude throughout the simulation: specifically, $0.5(1+0.5\sin(0.5t)) \in [0.25, 0.75]$. The additional temporal modulation is not intended to reproduce a specific benchmark forcing law, but was introduced as a synthetic extension to generate trajectories with mild time-dependent variation in the energy input while avoiding abrupt or high-frequency changes.

The dataset contains 1000 samples at a resolution of 256×256. For each sample, an initial vorticity field is generated in Fourier space. Fourier coefficients are assigned for indices $(k_x, k_y) = (i, j)$ with $i, j \in \{1, \ldots, k_{\max} - 1\}$ and $k_{\max} = 16$. These coefficients are random complex numbers whose amplitudes are modulated by a Gaussian envelope of the form $\exp\left(-\frac{1}{2}\frac{i^2+j^2}{(k_{\max}/2)^2}\right)$, which suppresses higher frequencies and yields smooth initial conditions. The coefficients in the $-k_x, -k_y$ quadrant are then set by conjugate symmetry so that the inverse Fourier transform produces a real-valued field. No coefficients are assigned in the mixed-sign quadrants. The initial vorticity field is finally obtained by applying an inverse Fourier transform, followed by normalization such that the maximum absolute value of the vorticity is 1.

Each initial condition is then evolved forward in time using a fourth-order Runge–Kutta (RK4) integration scheme to obtain the corresponding later vorticity field stored in the dataset.

This initialization procedure produces smooth but diverse initial conditions while concentrating most spectral energy in the low-frequency region. Figure B.1 illustrates this by showing the Fourier spectra of an example initial state and its corresponding final state. The figure highlights both the initially active modes and the effect of the Gaussian envelope, as well as the redistribution of spectral energy under the Navier-Stokes dynamics.

Appendix C

HTGR input engineering details

This appendix provides additional details on the input engineering used for the HTGR column temperature dataset, as introduced in Chapter 3. In this dataset, the model input consists only of the axial level z and the boundary temperature T. Since this input is low-dimensional, several alternative representations were tested to determine whether a more expressive encoding of the same information would improve predictive performance.

Table C.1 shows a representative subset of the tested input representations. The raw representation (z, T) already performs well, with a test relative RMSE of 9.10×10^{-3}. Normalizing the same two variables does not improve performance in this case, and instead gives a slightly higher test relative RMSE of 9.51×10^{-3}. This suggests that normalization alone is not necessarily beneficial for this dataset, and that the model may benefit from access to the original numerical scale of the axial position and boundary temperature.

The best-performing representation among the tested configurations is the normalized polynomial encoding (z, T, z^2, T^2, zT), which reduces the test relative RMSE to 9.46×10^{-4}. The added polynomial terms are not intended to represent new physical variables, and in particular z^2 and T^2 should not be interpreted as directly meaningful physical quantities for this problem. Instead, they provide the model with a richer representation of the simple scalar inputs, making certain nonlinear dependencies easier to learn. The fact that the basic normalized representation performs worse than the raw input, while the normalized polynomial representation performs best, also illustrates that the effect of feature engineering choices is not always straightforward.

A second engineered representation was tested using features with a clearer connection to the geometry of the problem: $(z, T, \sin(\pi(z - z_{\min})/L_z), T\sin(\pi(z - z_{\min})/L_z), \cos(\pi(z - z_{\min})/L_z))$. The sine terms vanish at the axial boundaries, while the cosine term equals one at the boundaries, giving the model access to features that encode boundary-aware structure in the z direction. This representation achieves a test relative RMSE of 2.58×10^{-3}, substantially improving over the raw representation but not matching the normalized polynomial model. It also converged in substantially less time in this experiment. This result suggests that physically motivated feature choices can be effective, but also that physical interpretability alone does not guarantee the best empirical performance.

Overall, these results show that the choice of input representation has a significant effect on the HTGR prediction task. The normalized polynomial representation performs best

Table C.1: Performance comparison of selected input representations for the HTGR column temperature dataset.

Input representation	Best val relRMSE	Epoch	Test relRMSE
(z, T)	8.623981×10^{-3}	30	9.102986×10
normalized (z, T)	1.018177×10^{-2}	19	9.513722×10
normalized (z, T, z^2, T^2, zT)	9.223184×10^{-4}	232	9.462218×10
$(z, T, \sin(\pi(z - z_{\min})/L_z), T\sin(\pi(z - z_{\min})/L_z), \cos(\pi(z - z_{\min})/L_z))$	2.472064×10^{-3}	84	2.579207×10

among the configurations shown here, while the boundary-aware trigonometric representation provides a useful comparison: it is physically easier to motivate and improves over the raw input, but does not reach the same accuracy as the best polynomial encoding.

Appendix D

Hyperparameters

This appendix provides a complete list of the hyperparameters used for each model and dataset in the experiments described in Chapter 3. The hyperparameters include architectural choices, training settings, and any dataset-specific parameters that were relevant for model performance. The listed hyperpameters were determined through a combination of prior literature, preliminary experiments, and manual tuning to achieve good performance.

Appendix E

Requirements and Guidelines

This chapter details some requirements and guidelines for MSc theses submitted to the Software Engineering Research Group.

E.1 Requirements

E.1.1 Layout

- Your thesis should contain the formal title pages included in this document (the page with the TU Delft logo and the one that contains the abstract, student id and thesis committee). Usually there is also a cover page containing the thesis title and the author (this document has one) but this can be omitted if desired.
- Base font should be an 11 point serif font (such as Times, New Century Schoolbook or Computer Modern). Do not use sans-serif fonts such as Arial or Helvetica. *Sans-serif type is intrinsically less legible than seriffed type* [11].
- The final thesis and drafts submitted for reviewing should be printed double-sided on A4 paper.

E.1.2 Content

- The thesis should contain the following chapters:
  - Introduction.
    Describes project context, goals and your research question(s). In addition it contains an overview of how (the remainder of) your thesis is structured.
  - One or (usually) more "main" chapters.
    Present your work, the experiments conducted, tool(s) developed, case study performed, etc.
  - Overview of Related Work
    Discusses scientific literature related to your work and describes how those approaches differ from what you did.

- Discussion/Evaluation/Reflection
  What went well, what went less well, what can be improved?
- Conclusions, Contributions, and (Recommendations for) Future Work
- Bibliography

E.1.3 Bibliography

- Make sure you've included all required data such as journal, conference, publisher, editor and page-numbers. When you're using BIBTEX, this means that it won't complain when running `bibtex your-main-tex-file`.
- Make sure you use proper bibliographic references. This especially means that you should avoid references that **only** point at a website and not at a printed publication.

  For example, it's OK to add a URL with the entry for an article describing a tool to point at its homepage, but it's not OK to just use the URL and not mention the article.

E.2 Guidelines

- The main chapters of a typical thesis contain approximately 50 pages.
- A typical thesis contains approximately 50 bibliographic references.
- Make sure your thesis structure is balanced (check this in the table of contents).

  Typically the main chapters should be of equal length. If they aren't, you might want to revise your structure by merging or splitting some chapters/sections.

  In addition, the (sub)section hierarchies with the chapters should typically be balanced and of similar depth. If one or more are much deeper nested than others in the same chapter this generally signals structuring problems.
- Whenever you submit a draft of your thesis to your supervisor for reviewing, make sure that you have checked the spelling and grammar. Moreover, *read it yourself at least once from start to end, before submitting to your supervisor*.

  Your supervisor is not a spelling/grammar checker!
- Whenever you submit a second draft, include a short text which describes the changes w.r.t. the previous version.